

Project Documentation

1. Introduction

Project Title: AI FAQ ASSISTANT API

Team Leader: S.pooja shree

Team Members:

- P.Mithra
- V.Vinitha
- S.Divya

2. Project Overview

Purpose: Build a robust, secure, and intelligent REST API platform to streamline FAQ creation, user authentication, and AI-powered content management for customer support operations.

Key Features:

- JWT Authentication & Authorization for secure private endpoints
- AI FAQ Generation using Google Gemini AI (questions, answers, topics)
- Semantic / Keyword-based FAQ Search for fast information retrieval
- Complete CRUD operations for FAQ records
- Category management for structured organization
- Password encryption with bcrypt
- Centralized error handling and Mongoose schema validation
- RESTful API architecture with MongoDB + Mongoose ODM

3. Architecture

Backend Stack: Node.js + Express.js

Database: MongoDB with Mongoose ODM (Users, FAQs)

AI Integration: Google Gemini SDK for automated FAQ generation

Security: JWT tokens + bcrypt password hashing + auth middleware

Architecture (MVC):

- **Model Layer** — Mongoose schemas with field validation and password hashing hooks
- **Controller Layer** — Request handling, business logic, delegation to services/models
- **View / Routing Layer** — Express routes mapping HTTP methods to controllers (headless API)
- **Middleware** — Auth token validation, centralized error handling, CORS

4. Software & Hardware Requirements

Software:

- OS: Windows 10/11, macOS, or Linux
- Node.js v18+, npm v9+, Express.js, MongoDB / Mongoose, Postman or Thunder Client
- Packages: express, mongoose, dotenv, cors, jsonwebtoken, bcrypt, @google/genai

Hardware: Intel Core i5 (8th Gen+) / AMD Ryzen 5, 8 GB RAM (16 GB recommended), 1 GB free disk space

5. Database Entities (MongoDB Collections)

User Entity

- `_id` (ObjectId), `name` (String, Required), `email` (String, Required, Unique)
- `password` (String, Hashed, Required), `role` (String, Default: "user"), `createdAt`, `updatedAt`

FAQ Entity

- `_id` (ObjectId), `question` (String, Required), `answer` (String, Required)
- `category` (String, Required), `createdBy` (FK → User), `createdAt`, `updatedAt`

AI Generation (Logical)

- `topic`, `generatedQuestion`, `generatedAnswer`, `generatedCategory`, `generatedAt` (produced by Gemini before persistence)

Category (Logical)

- `categoryName`, `description`, `totalFAQs`**Key Relationships:**
 - User → FAQ (One-to-Many): A user can create and manage multiple FAQ records
 - Category → FAQ (One-to-Many): Each category holds multiple FAQ documents
 - User → AI Generated FAQ (One-to-Many): Users can generate multiple AI-assisted FAQs

6. Roles and Responsibilities

Admin: Manage all users, view/update/delete any FAQ, monitor AI content, manage categories, full system access

Content Creator: Create FAQs, edit/delete own FAQs, generate FAQs with AI, organize under categories

Authenticated User: View public FAQs, search FAQs, generate AI-assisted answers, access personal profile

Public User: View published FAQs, search FAQs; cannot create/edit/delete; no access to protected endpoints

7. Setup Instructions

Prerequisites: Node.js v18+, MongoDB, npm, Git, Postman/Thunder Client **Installation**

Steps:

1. Create project folder and open in VS Code
2. Run: `npm init -y`
3. Install dependencies: `npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai`
4. Configure environment variables (`.env`) for MongoDB URI, JWT secret, Gemini API key
5. Start server: `npm run dev`

8. Folder Structure

```
AI-FAQ-Assistant-API/
├── src/
│   ├── server.js           # Main entry - env, DB, port binding
│   ├── app.js              # Express app setup, CORS, routes
│   ├── middleware/
│   │   ├── authMiddleware.js # JWT extraction & validation
│   │   └── errorMiddleware.js # Centralized error handling
│   └── models/
```

```
■ ■ ■ ■ User.js           # User schema + password hash hook
■ ■ ■ ■ FAQ.js           # FAQ schema with createdBy ref
■ ■ ■ ■ routes/
■ ■ ■ ■ authRoutes.js    # Register / Login
■ ■ ■ ■ faqRoutes.js     # Protected FAQ CRUD + search
■ ■ ■ ■ aiRoutes.js      # AI generate-faq endpoint
■ ■ ■ ■ controllers/     # Business logic handlers
■ ■ ■ ■ .env ■ ■ ■ ■ package.json
■ ■ ■ ■ README.md
```

9. API Documentation

Authentication (Public)

- **POST /api/auth/register** — Register new user (name, email, password)
- **POST /api/auth/login** — Login and receive JWT token

FAQ Operations (Protected – Bearer Token)

- **POST /api/faqs** — Create FAQ (question, answer, category)
- **GET /api/faqs** — List FAQs / public published articles
- **GET /api/faqs/search?q=keyword** — Semantic / keyword search (Public)
- **PUT /api/faqs/:id** — Update own FAQ
- **DELETE /api/faqs/:id** — Delete own FAQ

AI Automation (Protected)

- **POST /api/ai/generate-faq** — Generate FAQ pair from topic using Google Gemini

Payload example: { "topic": "Mongoose schema indexing validation" } **Sample Request**

Bodies:

Register:

```
{ "name": "Priya Sharma", "email": "priya@writeflow.com", "password": "securepassword123" } Create
```

FAQ:

```
{ "question": "How do I configure custom category tags?", "answer": "Navigate to settings...",
  "category": "Configuration" }
```

10. Authentication

JWT-based login system. On successful login the server returns a signed token. Protected routes require the header `Authorization: Bearer <token>`. Auth middleware extracts the token, verifies the signature, and attaches user context to the request. Passwords are hashed with `bcrypt` before storage. Only original authors (or admins) can modify their documents.

11. Running the Application

-
- Ensure MongoDB is running and `.env` is configured
 - Start the development server: `npm run dev`
 - Test endpoints using Postman or Thunder Client
 - Typical base URL: `http://localhost:5000` (or configured PORT)

12. Scenario & User Flow

Background: Priya, a customer support operations manager, struggles with repetitive inquiries, organizing entries, [classifying topics](#), and [tracking question trends](#).

Solution: The AI FAQ Assistant API provides a centralized, secure backend that decouples business logic from frontend systems. Authenticated creators register, generate AI-assisted FAQs, draft official schemas, and manage their records. Public users can browse and search published FAQs.

Outcome: Operational delays are minimized. Schema validation maintains data integrity, JWT tokens ensure only original authors modify documents, and indexing enables real-time discoverability.

13. Testing

- **Unit Testing** — Verified individual modules (Auth, FAQ CRUD, AI generation, Search)
- **Integration Testing** — Checked smooth interaction between routes, middleware, models and MongoDB
- **API Testing** — Validated all endpoints (register, login, create/search FAQ, AI generate) via Postman
- **Security Testing** — Confirmed protected routes reject unauthenticated requests; password hashing works
- **Result** — Application runs successfully with correct data flow and secure access control

14. Demo & Resources

-
- **API Testing Video:** https://drive.google.com/file/d/1u_w1sigiOynhmqTDbj4NBm3CLIMunEWN/view?usp=sharing
 - **Demo Link:** <https://drive.google.com/file/d/1pNK9mLFxJ2DKDK-G1h69pS4IEZGOis0u/view?usp=drivesdk>
 - **Code Drive:** https://drive.google.com/drive/folders/1aUqV5q25fE_iUPwWcLt02eJ-v-ytxpjz?usp=sharing

15. Known Issues

-
- Currently a pure backend REST API — no built-in frontend UI
 - AI generation depends on external Google Gemini API availability and quota
 - Semantic search is primarily keyword-based; advanced vector search not yet implemented
 - Role-based access control is present but advanced admin dashboard features are limited
 - No multi-tenant / multi-organization isolation in the current schema

16. Future Enhancements

-
- Advanced vector / embedding-based semantic search for better relevance
 - Frontend dashboard (React) for non-technical content creators
 - Multi-tenant support and organization-level FAQ isolation
 - Analytics on frequently asked questions and search trends
 - Rate limiting, API key management, and usage quotas
 - Webhook / notification support when new FAQs are published
 - Export FAQs to Markdown / PDF / knowledge-base formats
 - Fine-tuned domain-specific AI models for higher quality generation
-